

ESTACIÓN 2 · SEGUNDA ENTREGA

De cinco entregas semanales a un solo sistema integrado

Quantum Core — Programación orientada a objetos,
tolerancia a fallos, serialización y control de versiones

Informe técnico · Evidencia basada en el código y en la ejecución real del proyecto

ESTUDIANTE	Deibis Zuluaga Baena
PROGRAMA	Ingeniería de Sistemas · Facultad de Ingeniería
NÚCLEO	122618 Fundamentos de Software (FUNSO_2_N_H)
DOCENTE	Simón Peláez Loaiza
INSTITUCIÓN	CEIPA Business School
PAQUETE PRINCIPAL	quantum_core
DATOS DE PRUEBA	50 registros · 38 válidos, 12 descartados
FECHA DE ENTREGA	20 de septiembre de 2026

Documento de soporte para la entrega en Canvas · Normas APA 7.^a edición

De cinco entregas semanales a un solo sistema integrado: consolidación de Quantum Core con POO, tolerancia a fallos, serialización y control de versiones

Resumen

El informe documenta la consolidación de las cinco actividades de las Semanas 3, 4 y 5 en un único sistema de gestión de transacciones. El punto de partida no fue un conjunto de archivos correctos pero dispersos, sino un problema concreto que la acumulación dejó a la vista: la jerarquía de clases del dominio estaba escrita tres veces, una por actividad, de modo que cualquier cambio en una regla del negocio obligaba a modificar tres archivos y a confiar en que las tres copias siguieran diciendo lo mismo. La solución fue reorganizar el trabajo en el paquete `quantum_core`, donde las clases se definen una sola vez y cada capacidad —construcción de objetos, lectura tolerante a fallos, serialización y presentación— ocupa un módulo con una sola responsabilidad. Sobre un archivo de 50 registros con 12 fallos deliberados, el sistema carga 38 transacciones válidas, descarta 12 dejando constancia de la línea y el motivo de cada una, las serializa a JSON y las vuelve a construir conservando un monto de \$18.557.551,25 y la clase original de cada objeto. Dieciséis pruebas automáticas verifican cada una de esas afirmaciones, y el historial del repositorio documenta la construcción mediante trece *commits* y una rama de funcionalidad integrada por medio de un *pull request*.

Palabras clave: programación orientada a objetos, principios SOLID, manejo de excepciones, serialización JSON, control de versiones

01 · ALCANCE

Alcance de la entrega y componentes de la solución

La Estación 2 no corresponde a una semana en particular. Acumula la Actividad 1 de la Semana 3 (pilares de la programación orientada a objetos), la Actividad 2 de la misma semana (taller de principios SOLID), las dos actividades de la Semana 4 (recuperación con `try/except` y serialización en JSON) y la Actividad 1 de la Semana 5 (control de versiones con Git y GitHub). El enunciado pide presentar un repositorio profesional que cuente la historia de cómo se fue construyendo la solución, y no un conjunto de archivos sueltos.

OBJETIVO

Consolidar el sistema de gestión de transacciones aplicando POO, tolerancia a fallos y serialización, y evidenciar el proceso de construcción mediante el historial de versiones del repositorio.

La entrega se compone del paquete `quantum_core` con cinco módulos, un punto de entrada llamado `main.py`, dos archivos de datos de prueba, una batería de dieciséis pruebas automáticas y el presente informe. El repositorio es público y su historial forma parte de la evidencia. Todo el sistema se resuelve con la biblioteca estándar de Python: no se incorporó ninguna dependencia externa, de modo que el proyecto se ejecuta en cualquier instalación reciente del lenguaje sin instalación previa.

La duplicación heredada de las entregas semanales

Cada actividad se resolvió como un programa independiente, que era lo razonable mientras el objetivo era demostrar un concepto por semana. La consecuencia apareció al reunir el material: `diseño_pilares_poo.py`, `robustez_try_except.py` y `serializacion_json.py` contenían, cada uno, su propia definición de `TransaccionBase`, de las cuatro clases hijas y de la función que decide cuál construir. Tres copias del mismo modelo, mantenidas a mano.

El problema no es estético. Si mañana la comisión del débito dejara de ser fija, habría que corregir tres archivos; si se corrigiera solo uno, el sistema seguiría funcionando y entregando resultados distintos según el programa que se ejecutara. Esa clase de defecto no produce un error visible, sino un desacuerdo silencioso entre dos partes del mismo proyecto.

CRITERIO APLICADO

La duplicación no se corrige copiando mejor, sino eliminando las copias. El modelo pasa a estar definido en un solo lugar y los demás módulos lo importan.

El paquete `quantum_core` y la responsabilidad única

La reorganización toma el principio de responsabilidad única del taller de la Semana 3 y lo aplica ya no a una clase, sino a la arquitectura completa. Cada módulo del paquete tiene una sola razón para cambiar, y esa razón se puede enunciar en una frase.

TABLA 1

Módulos del paquete y su responsabilidad única

MÓDULO	RESPONSABILIDAD	ORIGEN
<code>modelo.py</code>	Define qué es una transacción y cómo se comporta. No lee archivos, no escribe registros de auditoría y no genera JSON.	Semana 3 · Act. 1
<code>fabrica.py</code>	Traduce el texto "CREDITO" que trae el archivo a la clase <code>TransaccionCredito</code> que corresponde.	Semana 3 · Act. 2
<code>cargador.py</code>	Lee texto plano y sobrevive a los registros corruptos dejando constancia de cada descarte.	Semana 4 · Act. 1
<code>serializador.py</code>	Traduce objetos a JSON y JSON a objetos, con los errores del proceso traducidos al lenguaje del dominio.	Semana 4 · Act. 2
<code>reportes.py</code>	Presenta los resultados en texto, JSON o CSV, sin intervenir en el cálculo.	Semana 3 · Act. 2

Nota. El módulo `main.py` no aparece porque no contiene reglas del negocio: únicamente llama a los demás en el orden correcto.

La dependencia entre módulos es dirigida y sin ciclos: fabrica conoce a modelo, cargador conoce a fabrica y serializador conoce a fabrica y a cargador. El modelo, que es la pieza más estable del sistema, no conoce a ninguno de los otros. Esa dirección importa: las reglas del negocio no dependen de la manera en que los datos entran o salen, de modo que cambiar el formato de entrada no obliga a tocar las clases.

04 · DISEÑO

Los tres pilares de la programación orientada a objetos

El encapsulamiento se concentra en el monto. El atributo real se llama `_monto` y el único camino para asignarlo es un *setter* que primero convierte el dato y después aplica la regla del negocio. Como el constructor asigna `self.monto = monto`, incluso el valor inicial pasa por esa validación: no existe forma de crear una transacción con un monto inválido.

FIGURA 1

El setter como único punto de entrada del monto

```
Visual Studio Code – quantum_core/modelo.py

49     # Getter: permite leer el monto con objeto.monto. El valor real vive en el
50     # atributo protegido _monto, al que el resto del programa no llega directo.
51     @property
52     def monto(self):
53         """Devuelve el monto protegido de la transaccion."""
54         return self._monto
55
56     @monto.setter
57     def monto(self, nuevo_monto):
58         """
59         Valida y almacena un nuevo monto.
60
61         De aqui salen DOS errores distintos, aunque los dos se llamen igual:
62
63         1. float(nuevo_monto) lanza ValueError cuando el dato es un texto que
64            no representa un numero, como "texto_invalido" o una cadena vacia.
65            Es un fallo de CONVERSION.
66         2. El raise lo lanzamos nosotros cuando el numero es negativo. El
67            dato si es numerico; lo que falla es la REGLA DE NEGOCIO.
68         """
69         nuevo_monto = float(nuevo_monto)
70
71         if nuevo_monto < 0:
72             raise ValueError("el monto no puede ser negativo")
73
74         # Se asigna a _monto para no volver a entrar al setter.
75         self._monto = nuevo_monto
```

Nota. Del mismo *setter* salen dos errores distintos con el mismo nombre: `float()` lanza `ValueError` por un fallo de conversión, y el `raise` se lo lanza el programa por un incumplimiento de la regla del negocio.

La herencia evita repetir ese trabajo. Las cuatro clases hijas reciben constructor, *getter*, *setter* y métodos de consulta sin escribir una sola línea adicional, y reescriben únicamente `calcular_impacto()`. Esa respuesta distinta al mismo método es el polimorfismo, y su efecto práctico se ve en el recorrido final del programa: el bucle no pregunta de qué clase es cada objeto, simplemente llama al método y Python elige la versión que corresponde.

FIGURA 2

Las cuatro clases hijas y su única diferencia

```
Visual Studio Code – quantum_core/modelo.py

113 # Todas reciben el constructor, el getter, el setter y los metodos de consulta
114 # de TransaccionBase. Cada una reescribe unicamente calcular_impacto(): esa
115 # respuesta distinta al mismo metodo es el polimorfismo.
116
117
118 class TransaccionCredito(TransaccionBase):
119     """Credito: el impacto corresponde al 2 % del monto."""
120
121     def calcular_impacto(self):
122         return self.monto * 0.02
123
124
125 class TransaccionDebito(TransaccionBase):
126     """Debito: comision fija de 1.500, sin importar el monto."""
127
128     def calcular_impacto(self):
129         return 1500.0
130
131
132 class TransaccionEfectivo(TransaccionBase):
133     """Efectivo: el impacto corresponde al 1 % del monto."""
134
135     def calcular_impacto(self):
136         return self.monto * 0.01
137
138
139 class TransaccionCripto(TransaccionBase):
140     """Cripto: el impacto corresponde al 3 % del monto."""
141
142     def calcular_impacto(self):
143         return self.monto * 0.03
```

Nota. Cada clase aporta una regla de impacto y nada más. Todo lo demás es heredado.

05 · SOLID

Del if/elif al registro de tipos

En las entregas semanales, la función que decide qué clase construir era una cadena de condiciones. Funcionaba, pero para admitir un tipo nuevo había que entrar a modificar una función que ya estaba probada. El principio abierto/cerrado sostiene que un módulo debe poder extenderse sin modificarse, y una cadena de `if / elif` obliga justamente a lo contrario.

La versión consolidada traslada esa decisión a un diccionario. La fábrica consulta el registro, y si el tipo no está, lanza `ValueError` con el nombre recibido. Agregar un tipo es agregar una entrada desde fuera, con una llamada a `registrar_tipo()`.

FIGURA 3

El registro de tipos que reemplaza la cadena de condiciones

```
Visual Studio Code – quantum_core/fabrica.py

42 # REGISTRO DE TIPOS: la tabla que reemplaza la cadena de if / elif
43 # =====
44 _REGISTRO = {
45     "CREDITO": TransaccionCredito,
46     "DEBITO": TransaccionDebito,
47     "EFECTIVO": TransaccionEfectivo,
48     "CRIPTO": TransaccionCripto,
49 }
50
51
52 def registrar_tipo(nombre, clase):
53     """
54     Agrega un tipo nuevo al registro sin modificar la fabrica.
55
56     La clase debe heredar de TransaccionBase; de lo contrario no tendria el
57     setter que valida el monto ni el metodo calcular_impacto(), y el resto del
58     sistema fallaria mas adelante, cuando ya seria dificil saber por que.
59     """
60     if not isinstance(clase, TransaccionBase):
61         raise TypeError("La clase registrada debe heredar de TransaccionBase.")
62
63     _REGISTRO[nombre.upper()] = clase
```

Nota. registrar_tipo() verifica que la clase herede de TransaccionBase. Sin esa comprobación se podría registrar cualquier objeto y el fallo aparecería mucho después, cuando ya sería difícil rastrear su origen.

Es el mismo razonamiento del taller de SOLID, donde agregar un cuarto formato de reporte consistió en escribir una clase hija sin tocar las anteriores. La diferencia es que ahora el principio no se ilustra en un ejercicio aparte, sino que gobierna la pieza por la que pasan todos los registros del sistema. El módulo reportes.py conserva esa misma estructura: ReporteTexto, ReporteJSON y ReporteCSV heredan de ReporteBase y el programa los usa sin distinguirlos.

06 · CONFIABILIDAD

Tolerancia a fallos y registro de auditoría

El blindaje logrado en la Semana 3 tiene una consecuencia incómoda: si nadie atrapa el error que lanza el objeto, el programa se detiene en el primer registro corrupto y pierde todos los que venían detrás. La recuperación consiste en atrapar ese error en el punto donde los datos crudos se convierten en objetos, dejar constancia y seguir.

La decisión de diseño está en la ubicación del bloque: el try va dentro del bucle, no alrededor. Así cada línea queda protegida por separado. Si envolviera todo el recorrido, el primer fallo seguiría costando el archivo completo.

FIGURA 4

El bloque try dentro del bucle, con un except por cada tipo de error

```
Visual Studio Code – quantum_core/cargador.py

138     for numero_linea, linea in enumerate(archivo, start=1):
139         linea = linea.strip()
140
141         # Las lineas vacias no son un error: no hay nada que procesar.
142         if not linea:
143             continue
144
145         # Se limpia cada campo por separado para tolerar espacios sobrantes
146         # alrededor de las comas, frecuentes en archivos reales.
147         datos = [dato.strip() for dato in linea.split(",")]
148
149         try:
150             # Esta es la unica linea que puede fallar, y por eso es la que
151             # va dentro del try: aqui los datos crudos se convierten en un
152             # objeto. El monto se entrega tal como viene del archivo, para
153             # que sea el setter de la clase el que intente convertirlo y
154             # aplique la regla del negativo.
155             validas.append(crear_transaccion(*datos))
156
157             # Caso 1: el dato es del tipo correcto pero su valor no sirve
158             # (texto no numerico, monto vacio, monto negativo o tipo
159             # desconocido). Se usa warning y no error porque el programa no
160             # esta fallando: detecta el dato malo, lo aparta y sigue.
161             except ValueError as error:
162                 fallos["ValueError"] += 1
163                 registro.warning(
164                     f"[Linea {numero_linea:>2}] ValueError: {error}"
165                     f" -> registro descartado: {linea}"
166                 )
167
168             # Caso 2: la cantidad de datos no es la esperada. Faltan o sobran
169             # columnas, asi que la llamada al constructor queda mal armada.
170             except TypeError:
171                 fallos["TypeError"] += 1
172                 registro.warning(
173                     f"[Linea {numero_linea:>2}] TypeError: cantidad de datos"
174                     f" incorrecta -> registro descartado: {linea}"
175                 )
```

Nota. No se usa except Exception. Atrapar todo esconde defectos reales del programa y convierte un error de programación en un registro descartado más.

TABLA 2

Condiciones de error que el sistema reconoce

EXCEPCIÓN	CAUSA	EJEMPLO DEL ARCHIVO	RESPUESTA
ValueError	El valor no se puede convertir a número	C003, DEBITO, texto_invalido	Descarta y continúa
ValueError	Es un número válido, pero incumple la regla del negocio	C004, CREDITO, -200000	Descarta y continúa
ValueError	El tipo de transacción no está registrado	C019, TRANSFERENCIA, 540000	Descarta y continúa
TypeError	Faltan columnas en el registro	C006, DEBITO	Descarta y continúa
TypeError	Sobran columnas en el registro	C035, CRIPTO, 1540000, EXTRA	Descarta y continúa
FileNotFoundError	No existe el archivo de entrada	--archivo datos/ no_existe.txt	Registra y detiene

Nota. La última fila es la excepción a la regla: sin datos de entrada no hay nada que recuperar, de modo que el proceso se detiene en lugar de simular que continúa.

Los descartes se registran con nivel warning y no con error, porque el sistema no está fallando: detecta un dato malo, lo aparta y sigue. El nivel error queda reservado para las condiciones que impiden continuar. Esa distinción es la que permite, más adelante, revisar el archivo de auditoría y responder cuántos registros se perdieron sin confundir un dato sucio con una falla del programa.

FIGURA 5

Carga del archivo con 50 registros y 12 fallos deliberados

```
Terminal - main.py · Etapa 1

$ python3 main.py

=====
ETAPA 1 · CARGA TOLERANTE A FALLOS (datos/transacciones_corruptas.txt)
=====
INFO | Inicio de carga del archivo datos/transacciones_corruptas.txt
WARNING | [Linea 3] ValueError: could not convert string to float: 'texto_invalido' -> registro descartado:
C003,DEBITO,texto_invalido
WARNING | [Linea 4] ValueError: el monto no puede ser negativo -> registro descartado: C004,CREDITO,-200000
WARNING | [Linea 6] TypeError: cantidad de datos incorrecta -> registro descartado: C006,DEBITO
WARNING | [Linea 13] ValueError: el monto no puede ser negativo -> registro descartado: C013,CRIPTO,-75000
WARNING | [Linea 15] ValueError: could not convert string to float: 'abc123' -> registro descartado: C015,CREDITO,abc123
WARNING | [Linea 19] ValueError: tipo de transaccion desconocido: TRANSFERENCIA -> registro descartado:
C019,TRANSFERENCIA,540000
WARNING | [Linea 22] ValueError: could not convert string to float: '' -> registro descartado: C022,EFFECTIVO,
WARNING | [Linea 28] TypeError: cantidad de datos incorrecta -> registro descartado: C027,CRIPTO
WARNING | [Linea 33] ValueError: el monto no puede ser negativo -> registro descartado: C032,DEBITO,-1
WARNING | [Linea 36] TypeError: cantidad de datos incorrecta -> registro descartado: C035,CRIPTO,1540000,EXTRA
WARNING | [Linea 43] ValueError: tipo de transaccion desconocido: CHEQUE -> registro descartado: C042,CHEQUE,210000
WARNING | [Linea 48] TypeError: cantidad de datos incorrecta -> registro descartado: C047
INFO | Fin de carga: 38 validas, 12 descartadas (8 ValueError, 4 TypeError)

Registros validos procesados : 38
Registros descartados : 12 (8 ValueError, 4 TypeError)
Monto total acumulado : $18,557,551.25
Impacto total calculado : $456,418.00
```

Nota. El mismo detalle, con fecha y hora en cada línea, queda escrito en `salidas/errores_carga.log`, que es el rastro auditable de la carga.

07 · INTEROPERABILIDAD

Serialización y persistencia en JSON

Los objetos viven en la memoria del programa: existen mientras se ejecuta y desaparecen cuando termina. Para guardarlos en disco o enviarlos a otro sistema hay que convertirlos en un formato de texto que cualquiera entienda. El recorrido tiene una escala intermedia que no es opcional: el módulo `json` no sabe traducir una clase propia del proyecto, pero sí sabe traducir un diccionario.

Al serializar se pierden los métodos, porque el archivo guarda únicamente los datos, y se pierde también la clase de origen, porque el JSON no la registra. Lo único que viaja es el texto del tipo. La reconstrucción aprovecha esa pista: la fábrica lee ese texto y decide qué clase construir, de modo que el objeto recuperado vuelve a tener su fórmula de impacto.

FIGURA 6

Reconstrucción del objeto con validación previa de las claves

```
Visual Studio Code – quantum_core/serializador.py

125 def diccionario_a_objeto(como_dict):
126     """
127     Reconstruye el objeto a partir de un diccionario.
128
129     Antes de llamar a la fabrica se comprueba que esten las tres claves
130     obligatorias. Sin esa validacion, un registro incompleto produciria un
131     KeyError, que no le dice a nadie cual fue el dato que faltó.
132     """
133     if not isinstance(como_dict, dict):
134         raise ValueError("el registro serializado no es un objeto JSON valido")
135
136     faltantes = [c for c in CLAVES_OBLIGATORIAS if c not in como_dict]
137
138     if faltantes:
139         raise ValueError(
140             "al registro le faltan las claves: " + ", ".join(faltantes)
141         )
142
143     # La fabrica decide la clase a partir del texto "tipo"; el setter de la
144     # clase valida el monto. La validacion no se repite aqui.
145     return crear_transaccion(
146         como_dict["cliente_id"], como_dict["tipo"], como_dict["monto"]
147     )
```

Nota. Sin la comprobación previa, un registro incompleto produciría un `KeyError`, que no informa cuál fue el dato que faltó. La validación traduce el fallo al lenguaje del dominio.

FIGURA 7

Fragmento del archivo generado por el programa

```
salidas/transacciones.json

[
  {
    "cliente_id": "C001",
    "tipo": "DEBITO",
    "monto": 150000.0,
    "impacto": 1500.0
  },
  {
    "cliente_id": "C002",
    "tipo": "CREDITO",
    "monto": 500000.0,
    "impacto": 10000.0
  },
  ... 36 registros más
]
```

Nota. El campo `impacto` se incluye porque es información útil para quien reciba el archivo y no se puede recalcular sin conocer la clase de origen.

El mismo criterio de recuperación de la lectura de texto plano se aplica aquí: un registro mal formado se descarta con su motivo y la deserialización continúa con los siguientes. Los fallos técnicos del módulo `json` se traducen a

ValueError con un mensaje del dominio, porque quien llama necesita saber qué dato está mal, no en qué línea de una biblioteca ocurrió el problema.

08 · VERIFICACIÓN

La prueba de que la integración funciona

Que cada pieza funcione por separado no demuestra que el conjunto funcione. La cuarta etapa del programa existe para responder esa pregunta: reconstruye los objetos desde el archivo JSON y compara el monto total con el de la carga original. Si los dos números coinciden, ningún dato se perdió ni se deformó en el recorrido.

FIGURA 8

Serialización, reconstrucción y verificación de integridad

```
Terminal - main.py · Etapas 3 y 4

=====
ETAPA 3 · SERIALIZACION A JSON
=====
INFO    | Serializacion completada: 38 registros escritos en salidas/transacciones.json

=====
ETAPA 4 · DESERIALIZACION Y VERIFICACION DEL VIAJE COMPLETO
=====
INFO    | Deserializacion: 38 objetos reconstruidos, 0 descartados

Objetos antes de serializar      : 38
Objetos reconstruidos del JSON: 38 (0 descartados)
Monto original                   : $18,557,551.25
Monto recuperado                 : $18,557,551.25
Integridad de los datos          : VERIFICADA
Clase recuperada del primero     : TransaccionDebito
```

Nota. Que la última línea diga TransaccionDebito y no TransaccionBase demuestra que la fábrica recuperó la clase especializada a partir del texto guardado en el archivo.

El enunciado no pide pruebas automáticas. Se incluyeron porque son la forma de demostrar que lo que este informe afirma se puede comprobar sin leer una salida en pantalla. Cada grupo de pruebas corresponde a una afirmación concreta: que el encapsulamiento rechaza montos inválidos, que las cuatro clases calculan impactos distintos, que la fábrica admite tipos nuevos sin modificarse, que la carga procesa 38 registros y descarta 12, y que el viaje a JSON conserva el monto y la clase.

FIGURA 9

Ejecución de la batería de pruebas automáticas

```
Terminal – unittest

$ python3 -m unittest discover -s pruebas -t .

test_el_monto_se_guarda_como_numero (PruebaEncapsulamiento) ... ok
test_el_setter_tambien_protege_una_asignacion_posterior (PruebaEncapsulamiento) ... ok
test_monto_negativo_es_rechazado (PruebaEncapsulamiento) ... ok
test_monto_no_numerico_es_rechazado (PruebaEncapsulamiento) ... ok
test_construye_la_clase_que_corresponde (PruebaFabrica) ... ok
test_no_se_admite_una_clase_ajena_a_la_jerarquia (PruebaFabrica) ... ok
test_se_puede_agregar_un_tipo_sin_modificar_la_fabrica (PruebaFabrica) ... ok
test_un_tipo_desconocido_lanza_value_error (PruebaFabrica) ... ok
test_cada_clase_calcula_su_propio_impacto (PruebaPolimorfismo) ... ok
test_la_clase_base_no_define_una_formula (PruebaPolimorfismo) ... ok
test_el_monto_total_sobrevive_al_viaje (PruebaSerializacion) ... ok
test_la_clase_se_recupera_desde_el_texto_del_tipo (PruebaSerializacion) ... ok
test_un_registro_incompleto_se_reporta_con_la_clave_que_falta (PruebaSerializacion) ... ok
test_un_texto_json_mal_formado_se_traduce_a_value_error (PruebaSerializacion) ... ok
test_conteo_de_validas_y_descartadas (PruebaToleranciaAFallos) ... ok
test_un_archivo_inexistente_detiene_el_proceso (PruebaToleranciaAFallos) ... ok

-----
Ran 16 tests in 0.024s

OK
```

Nota. Las pruebas detectaron un defecto real durante la construcción: al reconfigurar el registro de auditoría, los destinos anteriores se descartaban sin cerrarse. La corrección quedó documentada en el historial del repositorio.

09 · CICLO DE VIDA

Control de versiones como evidencia del proceso

El repositorio no se creó al final para cumplir con el requisito. Se inicializó antes de escribir la primera clase y creció con el proyecto, de modo que su historial refleja el orden real en que se tomaron las decisiones: primero el modelo, después la fábrica, luego los datos de prueba, la carga tolerante a fallos, la serialización, la presentación y el punto de entrada.

Los mensajes de *commit* explican por qué cambió cada cosa y no solo qué archivo se tocó. «Reemplaza la cadena if/elif de la fábrica por un registro de tipos» permite reconstruir el razonamiento meses después; «cambios» no permite nada.

FIGURA 10

Historial de construcción y estrategia de ramas

```
Terminal - git

$ git log --oneline --graph --all --decorate

* cffe541 (HEAD -> main) Cierra los destinos del log antes de reemplazarlos
* e83f0b6 Agrega el README con la documentacion completa del sistema
| * 799b8dc (feature/pruebas-de-integracion) Agrega la bateria de pruebas automaticas
| * b16b4bd Agrega eliminar_tipo() como operacion simetrica del registro
|/
* d0140e9 Agrega el punto de entrada que integra las tres semanas
* 4985f9e Separa la presentacion de los resultados en el modulo de reportes
* 3d7a249 Agrega la serializacion JSON de ida y vuelta
* 570bed5 Agrega la carga tolerante a fallos con registro de auditoria
* 0a69582 Incorpora los archivos de datos de prueba de la Semana 4
* 9ea44d6 Reemplaza la cadena if/elif de la fabrica por un registro de tipos
* 8d1cfc8 Agrega el modelo de dominio con los tres pilares de la P00
* 5b95163 Inicializa el repositorio de la Estacion 2 con su .gitignore
```

Nota. La rama `feature/pruebas-de-integracion` aísla el trabajo de las pruebas hasta que está terminado. Su integración a `main` se hizo mediante un *pull request* en GitHub.

La estrategia de ramas separa la versión estable del trabajo en curso. La rama `main` conserva en todo momento un sistema que se ejecuta; la rama de funcionalidad permite equivocarse sin consecuencias. Esa separación no es un formalismo: durante el desarrollo de las pruebas fue necesario agregar una función al módulo de la fábrica, y poder hacerlo sin tocar la rama principal hasta tener el conjunto verificado es exactamente el beneficio que el flujo persigue.

10 · RESULTADOS

Resultados verificados

Todas las cifras de esta sección provienen de la ejecución real del programa sobre el archivo de prueba, no de una estimación.

REGISTROS PROCESADOS 38 de 50 líneas del archivo de prueba	REGISTROS DESCARTADOS 12 8 ValueError y 4 TypeError, cada uno con su línea	INTEGRIDAD DEL VIAJE \$0 de diferencia entre el monto original y el recuperado
---	---	---

TABLA 3

Indicadores de la ejecución y de la verificación

INDICADOR	RESULTADO
Registros del archivo de prueba	50
Transacciones válidas procesadas	38
Registros descartados	12 (8 ValueError, 4 TypeError)
Monto total acumulado	\$18.557.551,25
Impacto total calculado	\$456.418,00
Registros serializados a JSON	38
Objetos reconstruidos desde JSON	38 (0 descartados)
Diferencia entre monto original y recuperado	\$0,00
Clase recuperada en la deserialización	La original de cada objeto
Pruebas automáticas superadas	16 de 16
Commits del repositorio	13
Ramas de trabajo	2 (main y una rama de funcionalidad)
Dependencias externas	Ninguna

Nota. Los valores corresponden a la ejecución de `main.py` sobre `datos/transacciones_corruptas.txt` y a la ejecución de la batería de pruebas.

11 · CONCLUSIONES

Conclusiones

La arquitectura del sistema quedó en un diseño donde la validación ocurre dentro del objeto. El resto del programa no puede crear una transacción mal formada, no porque exista un acuerdo de no hacerlo, sino porque el único camino hacia el atributo pasa por un método que valida. Definir el modelo una sola vez eliminó además la posibilidad de que dos partes del proyecto sostengan reglas distintas sobre el mismo concepto.

La confiabilidad se comprobó sobre un archivo que falla en 12 de sus 50 registros y aun así se procesa completo. La diferencia entre detenerse y continuar no está en la cantidad de código, sino en dónde se ubica el bloque de protección: dentro del bucle, cada línea responde por sí misma. El registro de auditoría convierte esos descartes en información revisable, que es lo que distingue a un sistema tolerante a fallos de uno que simplemente ignora errores.

La interoperabilidad se verificó con un recorrido de ida y vuelta que conserva el monto al centavo y recupera la clase de cada objeto. Ese resultado no se afirma: se mide en la cuarta etapa del programa cada vez que se ejecuta, y una de las dieciséis pruebas automáticas lo comprueba de manera independiente.

El control de versiones aportó algo más que el cumplimiento de un requisito. Trabajar con *commits* pequeños obligó a terminar cada pieza antes de empezar la siguiente, y la rama de funcionalidad permitió incorporar las pruebas —y con ellas, la corrección de un defecto que las pruebas descubrieron— sin poner en riesgo una versión que ya funcionaba. El historial del repositorio es, en ese sentido, la parte del informe que no se puede redactar después.

Referencias

Chacon, S., & Straub, B. (2014). *Pro Git* (2.^a ed.). Apress.

Martin, R. C. (2008). *Clean code: A handbook of agile software craftsmanship*. Prentice Hall.

Martin, R. C. (2017). *Clean architecture: A craftsman's guide to software structure and design*. Prentice Hall.

Python Software Foundation. (2026). *The Python standard library: json — JSON encoder and decoder*. <https://docs.python.org/3/library/json.html>

Python Software Foundation. (2026). *The Python standard library: logging — Logging facility for Python*. <https://docs.python.org/3/library/logging.html>

Van Rossum, G., & Drake, F. L. (2009). *Python 3 reference manual*. CreateSpace.